

NAME: C. PRIYANKA

REG NO: 192511074

14) Alpha-Beta Pruning Algorithm for Gaming

Aim

To write and implement a Python program for the Alpha-Beta Pruning algorithm to determine the optimal move in a two-player gaming environment by reducing the number of nodes evaluated in the Minimax search tree.

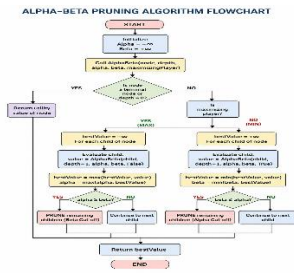
Objectives

- To understand the concept of the Minimax algorithm used in game playing.
- To implement Alpha-Beta Pruning using Python.
- To find the best possible move for the MAX player.
- To reduce unnecessary searches in the game tree.
- To improve the efficiency of the Minimax algorithm.

Algorithm

1. Start with the root node of the game tree.
2. Set:
3. $\alpha = -\infty$
4. $\beta = +\infty$
5. If the current node is a terminal node, return its utility value.
6. If it is a MAX node:
7. Initialize the best value to $-\infty$.
8. Evaluate each child node recursively.
9. Update α .
10. If $\alpha \geq \beta$, stop evaluating the remaining children (pruning).
11. If it is a MIN node:
12. Initialize the best value to $+\infty$.
13. Evaluate each child node recursively.
14. Update β .
15. If $\beta \leq \alpha$, stop evaluating the remaining children (pruning).
16. Return the best value.
17. The move having the highest value is selected by MAX.
18. Stop.

Flow Chart



Python Program

Alpha-Beta Pruning Algorithm

```
def alpha_beta(node, depth, alpha, beta, maximizing_player):
```

```
    # If leaf node, return its value
    if depth == 0:
        return node
```

```
    if maximizing_player:
        best_value = float('-inf')
```

```
    for child in node:
        value = alpha_beta(
            child,
            depth - 1,
            alpha,
            beta,
            False
        )
```

```
        best_value = max(best_value, value)
        alpha = max(alpha, best_value)
```

```
    # Alpha-Beta Pruning
    if alpha >= beta:
        break
```

```
    return best_value
```

```
else:
    best_value = float('inf')
```

```

for child in node:
    value = alpha_beta(
        child,
        depth - 1,
        alpha,
        beta,
        True
    )

    best_value = min(best_value, value)
    beta = min(beta, best_value)

# Alpha-Beta Pruning
if beta <= alpha:
    break

return best_value

```

```

# Game tree
game_tree = [
    [
        [3, 5],
        [6, 9]
    ],
    [
        [1, 2],
        [0, -1]
    ]
]

```

```

# Calculate depth
depth = 3

```

```

# Initial Alpha and Beta values
alpha = float('-inf')
beta = float('inf')

```

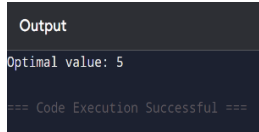
```

# Find optimal value
optimal_value = alpha_beta(
    game_tree,
    depth,

```

```
    alpha,  
    beta,  
    True  
)  
  
print("Optimal value:", optimal_value)
```

Sample Output



```
Output  
Optimal value: 5  
=== Code Execution Successful ===
```

Result

The Alpha-Beta Pruning algorithm was successfully implemented in Python. The program evaluated the game tree and determined the optimal value as:

Optimal value = 5

The algorithm reduces the number of nodes that need to be evaluated compared with the standard Minimax algorithm.

Conclusion

Thus, the **Alpha-Beta Pruning algorithm** was successfully implemented using Python for a gaming environment. It helps the AI player select the optimal move by eliminating unnecessary branches of the game tree. This makes game-playing algorithms **faster and more efficient** while producing the same optimal result as Minimax.